APCS 程式設計系列 · U06 · 實作中級 · C++ / Python 對照版

字串處理 重點講義

字元與 ASCII · 讀取整行 · 走訪與計數 · 子字串與切割 · 回文、翻轉與字典序 · 凱薩加密 · 大數加法

覺得有幫助？幫老師一個忙

- **訂閱** YouTube「莊老數學教室」：youtube.com/@chuanglaomath
- 看到老師的影片，順手**按個讚**
- 把這份免費講義**分享給同學**：wade-chuang.github.io/jenfoo/apcs（不用註冊，直接下載）

1. 字元就是整數：ASCII 碼

C/C++ 的 char 本身就是整數（ASCII 碼），可以直接加減；Python 的字元是字串，要用 ord(c) 轉成 ASCII 碼、chr(n) 轉回字元。兩邊的數值完全一樣。

字元	ASCII 碼	記法
空白 ' '	32	空白也是一個字元
'0' ~ '9'	48 ~ 57	數字字元 - '0' = 數值
'A' ~ 'Z'	65 ~ 90	大寫排在小寫前面
'a' ~ 'z'	97 ~ 122	小寫 = 大寫 + 32

C++ 字元與 ASCII 碼互換

```cpp
char c = 'C';
cout << (int)c << " ";          // 67：字元的 ASCII 碼
cout << (char)(c + 32) << " ";  // c：大寫 + 32 = 小寫
cout << '7' - '0' << "\n";      // 7：數字字元變數值
```

Python

```python
c = 'C'
print(ord(c), chr(ord(c) + 32), ord('7') - ord('0'))
# 67：ASCII 碼；c：大寫 + 32 = 小寫；7：數字字元變數值
```

輸入	輸出
（不需要輸入）	`67 c 7`

陷阱提醒 字元和數字別搞混

【錯】C++ 的 cout << c + 1 印出數字（要轉 (char)）；Python 的 'a' + 1 直接出錯（要用 chr、ord）。
【錯】'7' 和 7 不一樣：字元 '7' 的值是 55。把數字字元當數值用之前，先減掉 '0'。
【對】判斷大寫直接拿字元比：C++ 寫 c >= 'A' && c <= 'Z'，Python 寫 'A' <= c <= 'Z'。

2. 讀取一整行：輸入裡有空白怎麼辦

寫法	讀到什麼
C++ 的 cin >> s、C 的 scanf("%s", s)	一個「單字」，碰到空白或換行就停
C++ 的 getline(cin, s)	一整行（含空白），不含最後的換行
Python 的 input()	一整行（含空白），不含最後的換行
Python 的 input().split()	把一行切成單字串列，空白全部丟掉

C++ 讀取含空白的一整行

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    string s;
    cin >> n;
    cin.ignore();          // 吃掉 n 後面的換行
    getline(cin, s);       // 讀取一整行（含空白）
    cout << n << " " << s.size() << " [" << s << "]\n";
}
```

Python

```python
n = int(input())
s = input()          # input() 本來就讀取一整行
print(n, len(s), "[" + s + "]")
```

輸入	輸出
3 hi there apcs	3 14 [hi there apcs]

陷阱提醒

【錯】C++：cin >> n 之後要先 cin.ignore() 再 getline，不然會讀到空行。Python：要保留空白就不能 split()。

【錯】題目說「一行文字」而且可能有空白，就不能用讀單字的寫法，只會讀到第一個字。

C 題本：字元陣列與結尾的 '\0'

- char s[20] = "APCS"; 實際存的是 'A' 'P' 'C' 'S' '\0'。**'\0'（ASCII 碼 0）標示字串結束。**
- strlen(s) 從頭數到 '\0' 為止（不含 '\0'）；sizeof(s) 是整個陣列的大小 20。
- 走訪寫法：for (i = 0; s[i] != '\0'; i++)。把 s[k] 改成 '\0'，字串就從那裡被截斷。
- 比較字串用 strcmp(a, b)：a 比較小回傳負數、相等回傳 0、a 比較大回傳正數。不能寫 a == b。

3. 走訪與計數：一個字元一個字元看

C++ 大小寫、數字、母音各有幾個

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    string s, v = "aeiouAEIOU";
    getline(cin, s);
    int up = 0, low = 0, dig = 0, vow = 0;
    for (char c : s) {
        if (c >= 'A' && c <= 'Z') up++;
        else if (c >= 'a' && c <= 'z') low++;
        else if (c >= '0' && c <= '9') dig++;
        if (v.find(c) != string::npos) vow++;   // c 在 v 裡面
    }
    cout << up << " " << low << " " << dig << " " << vow << "\n";
}
```

Python

```python
s = input()
up = low = dig = vow = 0
for c in s:
    if 'A' <= c <= 'Z':
        up += 1
    elif 'a' <= c <= 'z':
        low += 1
    elif '0' <= c <= '9':
        dig += 1
    if c in "aeiouAEIOU":
        vow += 1
print(up, low, dig, vow)
```

輸入	輸出
Hello APCS 2026, Go!	6 5 4 4

走訪字串的三個常用技巧

- **大小寫轉換**：C++ 寫 c - 'a' + 'A' 或 toupper(c)；Python 寫 c.upper()。
- **計數陣列**：C++ 的 cnt[c - 'a']++、Python 的 cnt[ord(c) - ord('a')] += 1，26 個字母各出現幾次。
- **內建判斷**：C 的 isupper(c)、isdigit(c)；Python 的 c.isupper()、c.isdigit()。

4. 子字串與切割

想做的事	C++	Python
從位置 a 起取 len 個字	s.substr(a, len)	s[a:a+len]
取位置 a 到 b - 1	s.substr(a, b - a)	s[a:b]（不含 b）
從位置 a 取到最後	s.substr(a)	s[a:]
找 t 第一次出現的位置	s.find(t)（找不到是 npos）	s.find(t)（找不到是 -1）
把一行切成單字	stringstream 搭配 >>	s.split()

C++ 子字串：substr 與切片

```cpp
string s = "programming";
cout << s.substr(3, 4) << " ";   // 從位置 3 起取 4 個字
cout << s.substr(7) << " ";      // 從位置 7 取到最後
cout << s.find("gram") << "\n";  // 第一次出現的位置
```

Python

```python
s = "programming"
print(s[3:7], s[7:], s.find("gram"))   # 切片不含終點
```

輸入	輸出
（不需要輸入）	gram ming 3

C++ 切出每個單字，找最長的

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    string line, w, best;
    getline(cin, line);
    stringstream ss(line);    // 把這一行當成輸入來源
    int cnt = 0;
    while (ss >> w) {         // 一次取出一個單字
        cnt++;
        if (w.size() > best.size()) best = w;
    }
    cout << cnt << " " << best << "\n";
}
```

Python

```python
words = input().split()   # 連續空白也會正確切開
best = ""
for w in words:
    if len(w) > len(best):
        best = w
print(len(words), best)
```

輸入	輸出
we love APCS so much	5 love

陷阱提醒

【錯】C++ 的 substr(a, len) 第二個參數是長度；Python 的 s[a:b] 第二個數是終點而且不含。

【錯】Python 的 split(" ") 遇到連續空白會切出空字串，用不給參數的 split() 才安全。

5. 回文、翻轉與字典序

C++ 回文判斷：頭尾往中間比

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    string s;
    cin >> s;
    bool ok = true;
    int i = 0, j = s.size() - 1;
    while (i < j) {
        if (s[i] != s[j]) { ok = false; break; }
        i++;
        j--;
    }
    cout << (ok ? "Yes" : "No") << "\n";
}
```

Python

```python
s = input()
ok = True
i, j = 0, len(s) - 1
while i < j:
    if s[i] != s[j]:
        ok = False
        break
    i += 1
    j -= 1
print("Yes" if ok else "No")   # 偷懶寫法：s == s[::-1]
```

輸入	輸出
level	Yes

C++ 翻轉與字典序比較

```cpp
string a = "stressed";
reverse(a.begin(), a.end());       // 原地翻轉
cout << a << " ";
cout << (string("Zoo") < "apple") << " ";
cout << (string("app") < "apple") << " ";
cout << (string("10") < "9") << "\n";
```

Python

```python
a = "stressed"
print(a[::-1], "Zoo" < "apple",
      "app" < "apple", "10" < "9")
```

輸入	輸出
（不需要輸入）	C++： desserts 1 1 1 Python： desserts True True True

字典序：字串怎麼比大小

- 從第一個字元開始比，**第一個不一樣的字元**決定大小（比的是 ASCII 碼）。
- 大寫字母全部排在小寫前面：'Z'（90）小於 'a'（97），所以 "Zoo" < "apple"。
- 一個是另一個的開頭（前綴）時，短的比較小："app" < "apple"。
- 數字字串比的是字元不是數值："10" < "9"。要比數值，先比長度，長度一樣再比字典序。

6. 字元平移：凱薩加密

C++ 凱薩加密：字母往後移 k 位

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int k;
    string s;
    cin >> k;
    cin.ignore();
    getline(cin, s);
    k = (k % 26 + 26) % 26;   // 負的 k 也變成 0~25
    for (char &c : s) {       // & 才能直接改字串裡的字元
        if (islower(c)) c = (c - 'a' + k) % 26 + 'a';
        else if (isupper(c)) c = (c - 'A' + k) % 26 + 'A';
    }
    cout << s << "\n";
}
```

Python

```python
k = int(input()) % 26    # Python 的 % 結果一定是 0~25
s = input()
t = ""
for c in s:
    if 'a' <= c <= 'z':
        c = chr((ord(c) - ord('a') + k) % 26 + ord('a'))
    elif 'A' <= c <= 'Z':
        c = chr((ord(c) - ord('A') + k) % 26 + ord('A'))
    t += c
print(t)
```

輸入	輸出
3 Hello, World xyz!	Khoor, Zruog abc!

陷阱提醒

【錯】忘了 % 26：'z' + 3 會跑出 '}' 這種符號。先減 'a' 變成 0~25，加完取餘數，再加回 'a'。

【錯】大寫、小寫要分開處理，標點和空白原封不動。

【錯】C++ 的 -1 % 26 是 -1，要寫 (k % 26 + 26) % 26；Python 的 -1 % 26 本來就是 25。

- 解密就是往回移 k 位，等於往後移 26 - k 位。

7. 數字字串：大數加法

C++ 的 long long 只到 19 位數；Python 的 int() 在 3.11 之後遇到超過 4300 位數的字串也會出錯。所以大數一律用字串存，模仿直式加法：**從個位開始**，每一位相加再加上進位。

C++ 大數加法：從個位開始的直式加法

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    string a, b, r;
    cin >> a >> b;
    int i = a.size() - 1, j = b.size() - 1, carry = 0;
    while (i >= 0 || j >= 0 || carry) {
        int d = carry;
        if (i >= 0) d += a[i--] - '0';
        if (j >= 0) d += b[j--] - '0';
        r += char('0' + d % 10);   // 先倒著存
        carry = d / 10;
    }
    reverse(r.begin(), r.end());
    cout << r << "\n";
}
```

Python

```python
a, b = input().split()
i, j, carry = len(a) - 1, len(b) - 1, 0
digits = []
while i >= 0 or j >= 0 or carry:
    d = carry
    if i >= 0:
        d += ord(a[i]) - ord('0')
        i -= 1
    if j >= 0:
        d += ord(b[j]) - ord('0')
        j -= 1
    digits.append(str(d % 10))   # 先倒著存
    carry = d // 10
print("".join(reversed(digits)))
```

輸入	輸出
99999999999999999999 1	100000000000000000000

- 迴圈條件要包含「還有進位」：999 + 1 最後還要多跑一圈，補上最前面的 1。
- 兩個數長度不同時，短的那個用完就當作 0。
- C++：減 '0' 變數值、加 '0' 變字元；Python：int(c) 和 str(d)。

讀完這份，你應該要會

- □ 背得出 '0' = 48、'A' = 65、'a' = 97，C++ 直接加減、Python 用 ord / chr
- □ C++ 會 getline（記得 cin.ignore()）、Python 分得清 input() 和 split()
- □ 會走訪字串統計大小寫、數字、母音，也會用計數陣列
- □ C++ 的 substr(a, len) 給長度、Python 的 s[a:b] 不含 b
- □ 會判斷回文；知道 "Zoo" < "apple"、"app" < "apple"、"10" < "9"
- □ 凱薩加密會用 % 26 繞回，也會處理往回移（負的 k）
- □ 大數加法會從個位開始逐位相加並處理最後的進位

有一項沒把握，就回去重看那一段，再做練習題。

看影片講解、問問題

YouTube 莊老數學教室　IG @jf_chuang　LINE 諮詢 jenfoo2021

覺得有幫助？幫老師一個忙

- **訂閱** YouTube「莊老數學教室」：youtube.com/@chuanglaomath
- 看到老師的影片，順手**按個讚**
- 把這份免費講義**分享給同學**：wade-chuang.github.io/jenfoo/apcs（不用註冊，直接下載）